

Guia practica - Metodologia CVE y Exploits para THM / CTF

Uso previsto: laboratorios autorizados, TryHackMe y CTF. Ajusta rutas, IPs, credenciales y herramientas segun el entorno.

Objetivo

Guia para pasar de 'veo una version' a 'tengo un exploit razonable' sin ejecutar cualquier PoC a ciegas.

En CTF el tiempo importa, pero un exploit incorrecto puede hacerte perder mas tiempo que una enumeracion cuidadosa.

1. Confirmar version real

```
nmap -sC -sV -p PUERTO $IP
curl -i $URL/
whatweb -a 3 $URL
searchsploit PRODUCT VERSION
banner grab: nc -nv $IP PUERTO
```

- No confies en un solo banner; puede estar oculto, parcheado o falseado.
- Busca version en headers, HTML, JS, changelog, readme, package files y rutas admin.
- Diferencia producto, plugin, tema, modulo y sistema operativo.

2. Triage de candidatos

Pregunta	Decision
Coincide producto exacto?	Si no, baja prioridad.
Coincide rango de versiones?	Si esta fuera, busca bypass o descarta.
Requiere credenciales?	Separar pre-auth de post-auth.
Requiere rol admin?	Solo util si ya tienes ese rol.
Es RCE, LFI, auth bypass o info leak?	Prioriza segun cadena de ataque.
Es destructivo?	Evitar salvo que el lab lo pida explicitamente.

3. Leer PoC antes de ejecutar

```
sed -n '1,220p' exploit.py
grep -nEi "rm|del|format|shutdown|useradd|chmod|curl|wget|socket|subprocess" exploit.py
python3 exploit.py -h
```

- Identifica inputs: RHOST, RPORT, URL, usuario, password, callback, archivo.
- Busca acciones peligrosas: rm, format, shutdown, useradd persistente, cambios irreversibles.
- Comprueba dependencias y version de Python/Ruby/Go.
- Entiende que condicion confirma exito: status, archivo creado, callback, salida en HTTP.

4. Adaptar exploit con control

Primero haz que imprima o confirme, luego que explote.

```
# 1. Probar reachability
curl -i $URL/vulnerable/path

# 2. Reproducir manualmente la parte minima
curl -i -X POST $URL/endpoint -d 'param=test'

# 3. Ejecutar PoC en modo check si existe
```

```
python3 exploit.py --check --url $URL
```

- Reduce el PoC a la petición esencial cuando puedas.
- Cambia payloads a comandos inocuos primero: id, whoami, hostname.
- Loguea requests/responses para poder explicar la cadena.

5. Compilacion segura de exploits

```
file exploit.c
grep -nEi "socket|connect|system|popen|exec|fork|unlink|remove" exploit.c
gcc exploit.c -o exploit
./exploit --help
```

- Compila en una carpeta temporal, no como root salvo que sea imprescindible.
- Si el exploit es para la víctima, compila para su arquitectura: x86/x64, Linux/Windows.
- En Windows labs, revisa si necesitas mingw: x86_64-w64-mingw32-gcc.

6. Cuando descartar

- La versión no coincide y no hay evidencia de vulnerabilidad.
- Requiere credenciales/rol que no tienes.
- El PoC depende de una configuración que el target no expone.
- Solo causa DoS y la room busca flags/acceso.
- No puedes explicar que hace después de leerlo.

7. Registro de evidencia

```
mkdir -p evidence exploit notes
cp exploit.py exploit/original_exploit.py
script -a evidence/exploit_run.log
# ejecutar prueba
exit
```

- Guarda versión detectada, fuente del exploit, cambios hechos y output de éxito.
- Anota precondiciones: usuario requerido, ruta, configuración, puerto.
- Si falla, registra error y razón probable para no repetir bucles.